GETRF: A General Framework for Encrypted Traffic Identification With Robust Representation Based on Datagram Structure

Xiaojuan Wang, Zikui Lu, Xinlei Wang, Mingshu He, *Member, IEEE*, and Xiaojun Wang

***Abstract*—Identifying encrypted traffic is essential for network management, but it becomes increasingly challenging as more and more users adopt encryption protocols and traffic defense mechanisms for data privacy. Most existing methods heavily rely on deep features and labeled samples, overlooking the opportunity to capture pattern information of datagram structure. As a consequence, these methods are unable to extract rule-based knowledge from unlabeled data, limiting their ability to identify unseen or feature-perturbed traffic. In this paper, we propose a new framework for encrypted traffic identification named Generic Encrypted Traffic Representation Framework (GETRF), which includes three pre-training tasks designed based on the datagram structure. By leveraging the advantages of these tasks, GETRF can utilize unlabeled traffic to form generic and robust representations, capturing the characteristics of datagrams at the byte, packet, and flow levels. After fine-tuning, these representations can be applied to identification tasks across multiple scenarios. The results demonstrate that GETRF achieves state-of-the-art performance in multi-scenario and fine-grained identifications, and effectively handles feature-perturbed traffic.**

***Index Terms*—Encrypted traffic identification, pre-training, generic traffic representation, cyberspace management, datagram structure, privacy-enhancing technology.**

I. Introduction

IDENTIFYING and categorizing traffic behaviors into their respective applications and services is crucial for cyberspace management [1], [2], [3], [4], [5]. They allow Internet service providers to improve the quality of service as well as detect potential security threats [6]. However, with the growth of privacy-enhancing technologies [7], [8], traditional methods and machine learning models are encountering two significant challenges. Firstly, the availability of plaintext is decreasing, making methods based on port numbers and protocol information unreliable. Secondly, the increasing randomness of payload data and statistic values makes fingerprinting and feature-counting methods inadequate for the current environment.

Deep learning, through hierarchical mapping and weighted computations, enables automatic feature extraction, thereby addressing the challenges posed by random algorithms [9], [10]. As such, researchers employ end-to-end models to classify encrypted traffic. However, these end-to-end methods heavily depend on the feature distribution of samples and label quality, which can limit the generalization ability of the models. To mitigate the impact of label quality on models, researchers begin to adopt pre-training approaches for identification. Works [11], [12] apply pre-training techniques to encrypted traffic classification and achieve significant results in application classification and attack detection tasks. Although reducing the amount of labeled data during the training, these works overlook the effect of latent information of datagram structure on traffic representation, resulting in poor performance in multi-scenario and fine-grained classification.

To better tackle the challenge of encrypted traffic identification, some practical issues that need to be discussed:

1) The method for encrypted traffic identification should minimize the use of specific field values and hand-craft features.

Encryption algorithms gradually cover plaintext, rendering the traffic behavior analysis based on specific values ineffective. Manual feature extraction, which often relies on expert knowledge, faces challenges due to the rapid growth of applications and services on the network, making the manual update of effective features increasingly complex.

2) The method for encrypted traffic identification should be robust and not rely on label quality or specific feature distributions.

Data labeling is a time-consuming task, and such a data collection process not only results in long bootstrap times but also poses additional challenges for identification. Furthermore, traffic defense mechanisms aim to disrupt the feature distribution, negatively impacting models that rely on the specific feature distribution of sample data for traffic classification.

3) The method for encrypted traffic identification should be generalized and able to be quickly applied to multi-scenario tasks as well as fine-grained classification.

Encrypted traffic identification includes application classification, attack detection, anonymous detection, encapsulation

Manuscript received 16 November 2023; revised 27 March 2024; accepted 7 May 2024. Date of publication 14 May 2024; date of current version 6 December 2024. This work was supported by the National Natural Science Foundation of China under Grant 62227805 and Grant 62071056. The associate editor coordinating the review of this article and approving it for publication was Y. Sagduyu. *(Corresponding authors: Zikui Lu; Mingshu He.)*

Xiaojuan Wang, Zikui Lu, Xinlei Wang, and Mingshu He are with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: Luzikui@bupt.edu.cn; hemingshu@bupt.edu.cn).

Xiaojun Wang is with the School of Electronic Engineering, Dublin City University, Dublin 9, Ireland.

Digital Object Identifier 10.1109/TCCN.2024.3400825

detection, malware detection, and other tasks. An ideal method should perform well across all of these tasks. Furthermore, in real network scenarios, there exists a significant amount of background traffic. A method with strong generalization capabilities should be able to mitigate the impact of background traffic and complete fine-grained identification of target flows.

To sum up, considering the application scenarios and challenges faced in encrypted traffic identification, an ideal method needs to satisfy both generality and robustness. For generality, employing pre-training models to generate traffic representations is a natural idea. These models can leverage large-scale unlabeled data and minimize the impact of specific feature distributions on generic representations by separating representation objectives from specific task objectives. Compared to generality, robustness is a more challenging issue. We consider that addressing this issue should focus on the datagram structure itself. Due to the limitations of current random algorithms and the cold-start policy followed by the traffic protection measures, privacy-enhanced technologies cannot produce perfectly random encrypted traffic. Therefore, similar patterns exist in datagrams of the same type. These similarities, observable in the traffic structure and padding characters, are influenced by the protocol, content, and encryption algorithm. By thoroughly leveraging this metadata, we can uncover latent patterns behind the traffic.

Based on the above considerations, in this paper, we propose a new framework with a pre-training model, named Generic Encrypted Traffic Representation Framework **(GETRF)**, for building generic and robust traffic representations. The framework includes three pre-training tasks based on datagram structure: Mask Token Prediction **(MTP)**, Same Packet Prediction **(SPP)**, and Hidden State Feature Prediction **(HSP)**. Benefiting from these tasks, GETRF can utilize unlabeled traffic to form generic and robust representations by capturing datagram characteristics at the byte, packet, and flow levels. Additionally, knowledge extraction from different aspects of datagram structures can also help mitigate the impact of traffic defense mechanisms. The main contributions of this paper are summarized as follows:

- We propose a novel pre-training learning framework for encrypted traffic identification, which leverages large-scale unlabeled data to achieve high-performance and multi-scenario traffic classification without model modification or extensive data training. Moreover, the framework can effectively mitigate interference of privacy-enhancing technologies and background traffic.
- We design three pre-training tasks based on datagram characteristics, capturing byte, packet, and flow-level traffic information to generate generic and robust representations. Benefiting from the knowledge extraction from different aspects of datagram structures, representations that are fine-tuned with a small amount of sample data can be applied to various tasks. Additionally, multi-perspective learning enhances the framework's ability to resist feature perturbations.
- We conduct comprehensive experiments to evaluate the framework using five public datasets and an experimental dataset. The results show that GETRF achieves state-of-the-art performance across five tasks with an average F1 score of 99.12%. Additionally, we further validate the ability of GETRF for fine-grained identification and its capability to resist traffic defense mechanisms, and our method shows effectiveness under these conditions.

The remainder of the paper is organized as follows: Section II discusses the related work. Section III proposes the framework architecture and pre-training tasks. Section IV presents the results and analysis of comparison experiments. Finally, the paper is concluded in Section V.

II. Related Work

A. Encrypted Traffic Identification

The research on traffic identification has been widely applied across a variety of applications. For discussion purposes, we categorize the methods into three classes [6], with the related works summarized in Table I.

1) Statistic Method: The earliest methods used for traffic classification were port-based methods and DPI [13]. However, these methods heavily relied on unencrypted information, and random changes in it could lead to misleading results. In contrast, our method learns the traffic representations from multiple perspectives, which helps to enhance its robustness.

2) Machine Learning Method: To reduce the impact of random content on classification, researchers utilize side-channel information and machine learning to classify traffic. Panchenko et al. [14] identified unknown traffic using features like packet size, packet direction, and the volume of bytes. AppScanner [15] trained classifiers using statistical features such as packet sizes, while BIND [16] exploited temporal features to classify traffic. However, these methods rely on expert knowledge and require appropriate feature design. The evolving nature of network traffic makes solutions, based on manually- and expert-originated features, unable to keep pace.

3) Deep Learning Method: Deep learning methods include end-to-end approaches, multimodal-based approaches, and representation-based approaches.

Wang et al. [17] first introduced the end-to-end method for flow classification, transforming packet data into grayscale images for CNN. Building on this, Sirinam et al. [18] employed CNN for deep fingerprinting, while Bhat et al. [19] designed Var-CNN for flow classification. However, these schemes did not fully consider the characteristics of datagram structure in their design, leading to unsatisfactory model performance. Lotfollahi et al. [20] developed DeepPacket to characterize raw payloads. Luo et al. [21] utilized transformer models to capture semantic relationships between characters for traffic classification. Shen et al. introduced GraphDApp [22], transforming application traffic into graphs for GNN-based identification. Huoh et al. [23] analyzed the relationships between traffic bytes, metadata, and packet interactions for application classification. Although these methods do not require feature engineering, they require substantial amounts of new, task-specific labeled data for retraining when addressing new tasks. In contrast, our model can be easily adapted to new scenarios with minimal fine-tuning on a small dataset.

TABLE I
NOTABLE RELATED WORKS IN ENCRYPTED TRAFFIC IDENTIFICATION TASKS

Category	Methods	Classifier	Feature	Raw Data	Utilization of Unlabeled Data	Robustness Verification	Application Scenario
Statistic method	DPI	-	packet size, packet direction, etc.	-	-	BT	AC,AD,MD,ND,ED
Machine learning method	CUMUL	SVM	packet size,packet, direction,etc.	-	-	BT	ND
	AppScanner	Random Forest,SVM	packet direction, packet numbers, etc.	-	-	-	AC
	BIND	SVM,Random Forest	packet size, uni-burst size,etc.	-	-	TDM	AC,ND
Deep learning mehtod	DF	CNN	pakcet direction	✓	-	TDM	AC,ED
	Var-CNN	CNN	packet direction, inter-packet time	✓	-	TDM	AC,ED
	DeepPacket	SAE+CNN	automatic extract	✓	-	-	AC,ED
	MIMETIC-ALL	CNN+BiGRU	automatic extract	✓	-	-	AC,ED
	APP-Net	CNN+BiLSTM	packet direction	✓	-	-	AC,ED
	GraphDApp	GNN	automatic extract		-	-	AC
	MIMETIC	CNN+GRU	inter-arrival time, packet direction, etc.	✓	-	-	AC
	ET-BERT	Attention+Transformer	automatic extract	✓	✓	-	AC,AD,MD,ND,ED
	GETRF	Attention+Transformer	automatic extract	✓	✓	BT,TDM	AC,AD,MD,ND,ED

[1]In the Robustness Verification columns, BT is background traffic interfence validation, TDM is traffic defense mechanisms validation.

[2] In the Application Scenario columns, AC is Application Classification, AD is Attack Detection, ND is Anonymous Detection, ED is Encapsulation Detection, MD is Malware Detection.

To overcome the limitation of single-model approaches, which fail to generalize high-dimensional features, researchers adopt multimodal approaches to classify traffic. MIMETIC [24], by assembling models that observe different features, addresses the problem of multi-view capitalization of traffic data. However, this method still requires a large number of labeled samples for training, resulting in a long bootstrap time. Considering bursts and communication protocols separately, Mimetic-All [25] leveraged context inputs as an additional modality for application classification, while APP-Net [26] combined BiLSTM and CNN models for identification. Both methods demonstrate high performance in application classification. Nevertheless, these approaches fail to take the influence of background traffic into account and cannot be easily applied to other classification scenarios.

Representation-based approaches leverage pre-training to utilize unlabeled data, which has been proven effective in NLP [27]. Similar to the "paragraph-sentence-word" structure in natural language, there is a "flow-packet-byte" hierarchy in network traffic. Therefore, researchers employ representation-based methods for traffic identification. He et al. [11] applied ALBERT to encrypted traffic classification and achieved 93.27% accuracy in ISCX-VPN [28]. However, the method did not design reasonable pre-training tasks, limiting the model's performance. Observing the process of Web page rendering, Lin et al. [12] designed a new structure named BURST for the pre-training task and generated traffic representation for identification. Nonetheless, this method did not consider the basic structure of the traffic, leading to poor performance in fine-grained identification. Our method, however, improves performance by learning the context, structure, and hidden state features of the flow, and maintains good performance when faced with feature-perturbed traffic.

B. Traffic Defense Mechanism

The purpose of traffic identification is to classify traffic and manage cyberspace effectively. However, some studies have demonstrated that traffic patterns can be distorted through methods such as dummy packet padding or traffic regularization [29], [30]. These methods, which prevent accurate traffic identification, are referred to as traffic defense mechanisms [31].

During the transmission of traffic, datagrams pass through multiple relay nodes. By deploying defense models at these relay nodes, traffic can be manipulated, resulting in distorted traffic observations at these nodes.

The primary methods of traffic defense mechanisms include obfuscation, confusion, and regularization [29]. Among these, both obfuscation and confusion methods involve dummy packet padding. This approach can significantly disrupt the feature distribution of traffic, making it difficult to distinguish between dummy packets and real packets. Moreover, it exploits the advantages of low overhead, making it feasible for implementation in the real world. For instance, WTF-PAD has been deployed in Tor circuits [30]. Regularization methods restrict how nodes send and receive packets to strictly limit the feature space, for example, by enforcing a fixed packet rate [31].

Given the potential for distortion in the traffic, we further evaluate the performance of GETRF on datasets processed by the traffic defense mechanism. It is worth noting that regularization methods are challenging to implement in the real world. Hence, we opt to manipulate the traffic by adding dummy packets for evaluation purposes.

III. THE PROPOSED METHOD

In this section, we present the design details of GETRF. We start with an overview of GETRF before exploring its individual components.

As mentioned above, to achieve a generic and robust traffic representation, two key considerations are necessary: 1) Designing pre-training tasks that separate representation goals from identification objectives, thereby preventing the overfitting of representations to specific feature distributions.

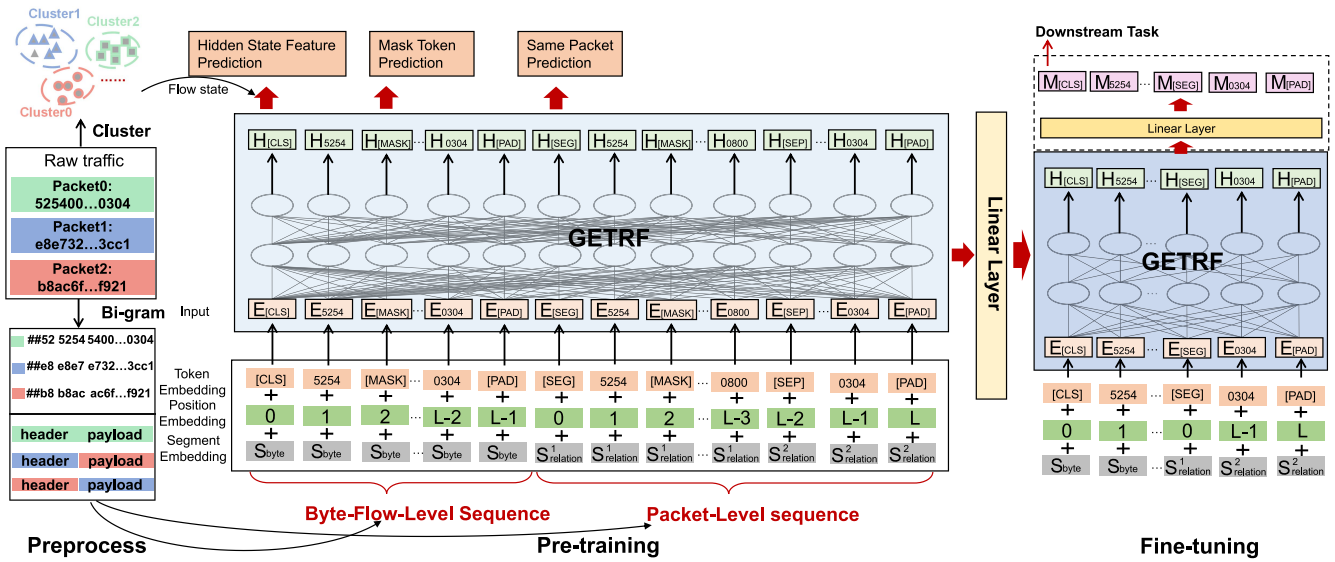


Fig. 1. Overview of the GETRF framework. The framework consists of three stages: preprocess, pre-training, and fine-tuning. In the preprocess stage, the raw traffic is processed to form tokens and construct the model's input sequence. In the pre-training stage, three pre-training tasks are utilized to contribute to generating the traffic representations. In the fine-tuning stage, these representations are adjusted based on the task-specific data for fine-grained traffic identification.

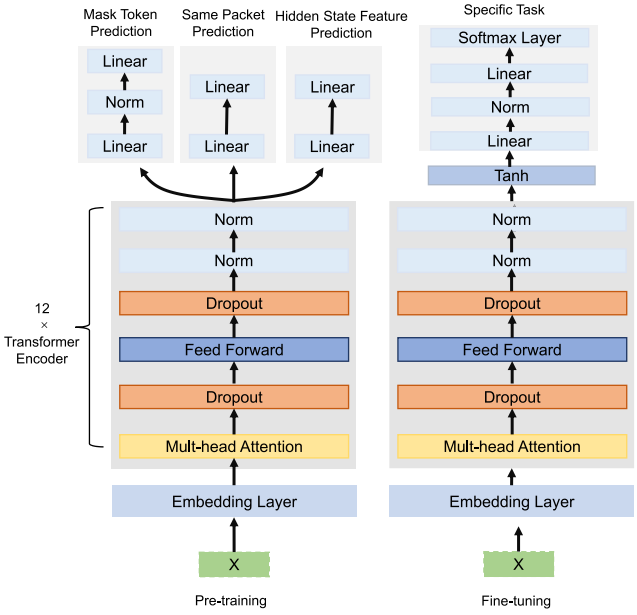


Fig. 2. The training process utilized during the pre-training and fine-tuning phases of GETRF. During the pre-training phase, the input sequences are processed by the embedding layer and the transformer encoder layer, resulting in the traffic representations. These representations are then fed into the three linear layers to acquire knowledge at different levels respectively. During the fine-tuning stage, the three networks associated with pre-training tasks are removed. Instead, an additional linear layer is added to perform specific identification tasks.

2) Minimizing the impact of feature perturbations on pre-training objectives. In other words, the representations should be resilient to changes in encrypted traffic without significant adjustments to their initial learning objective.

The design of pre-training tasks in GETRF is based on two key observations: Firstly, the raw bytes, metadata, and packet relationships contain more information that can be captured by the model [23], and specific-function metadata often clusters within traffic segments, forming distinct patterns [32]. Secondly, leveraging data from multiple dimensions can enhance model robustness [24]. Therefore, with a focus on traffic metadata, we design pre-training tasks based on datagram structure, namely MTP, SPP, and HSP. These tasks are designed to learn traffic representations at the byte, packet, and flow levels, respectively. The description of these three methods is provided in Section III-B.

We plot the architecture of GETRF in Fig. 1. At a high level, GETRF needs to analyze the correlation among three pre-training methods to generate representations. In particular, the transformer model with a self-attention mechanism is a reasonable choice. The self-attention mechanism establishes unique correlations between representations and patterns. Additionally, the multi-head mechanism further enables the framework to analyze comprehensive traffic patterns at three different levels. Therefore, we design the model as shown in Fig. 2 to capture traffic representations for subsequent training phases. Specifically, GETRF consists of an embedding layer, a transformer encoder layer, and three linear layers (or one linear layer). The embedding layer includes three embedding functions, which embed token, position and segment information of the input sequence, respectively. The transformer encoder layer includes 12 transformer encoders, each of which includes one multi-head attention model, one feed-forward model, two dropout functions, and two normalization functions. The multi-head attention model consists of four linear functions and one dropout function, while the feed-forward model consists of two linear functions. The selection of hyperparameters is provided in Table IV.

GETRF is composed of three main components. First, preprocessing the raw data to form token sequences suitable for the pre-training model, as detailed in Section III-A. Second, based on three pre-training tasks outlined in Section III-B, the

TABLE II
NOTATIONS USED IN THE PAPER

symbol	Description
T^r	The raw traffic
F	The collection of flows with the same 5-tuple
f_i	The i-th flow in collection of F
p_i^m	The m-th packet of the i-th flow, which contains a byte sequences
p_h^{mi}	The header of the packet p_i^m
p_p^{mi}	The payload of the packet p_i^m
b_j^{mi}	The j-th byte of the packet p_i^m
S_i^m	The token sentence corresponding to the packet p_i^m
S_h^{mi}	The token sentence corresponding to p_h^{mi}
S_p^{mi}	The token sentence corresponding to p_p^{mi}
L	The maximum length of token sequence
K_{bf}^{mi}	The byte-flow-level sequence
K_p^{mi}	The packet-level sequence
U^{mi}	The sequence fed into the pre-training model
Z_i	The average packet size of flow f_i
T_i	The logarithm of the average inter-packet interval time of the flow f_i
S_i	The hidden state feature of the flow f_i
h_i	The hidden state feature of S_i^m
$s(\cdot)$	Packet size
N	The number of flows
M	The number of packets in a flow
I	The number of bytes in a packet
θ	model parameters
t	The arriving time of a packet

encoding models are pre-trained, allowing for the acquisition of traffic representation using unlabeled data. Finally, for specific downstream tasks, fine-tuning is conducted with a small set of labeled data, as described in Section III-C. Table II provides a list of all symbols used in this paper.

A. Preprocess

The preprocess step aims to generate an appropriate input sequence for the execution of the pre-training tasks.

We first split the raw traffic T^r with the same 5-tuple (source IP, destination IP, source port, destination port, and protocol), and then form the collection of session flow *F*. Each session flow f_i represents a specific behavior and contains multiple packets, with packet p_i^m denoting the m-th packet of the i-th flow and b_j^{mi} denoting the j-th byte of the packet p_i^m. The relationship between flows, packets, and bytes is shown as the following formulas:

$$F = \{f_1, f_2, \ldots, f_i, \ldots, f_N, N \in \mathbb{N}^+\} \tag{1}$$

$$f_i = \{p_i^1, p_i^2, \ldots, p_i^m, \ldots, p_i^M, M \in \mathbb{N}^+\} \tag{2}$$

$$p_i^m = \{b_1^{mi}, b_2^{mi}, \ldots, b_I^{mi}, I \in \mathbb{N}^+\} \tag{3}$$

Even though both natural language and network traffic share a structure comprising 'words, sentences, paragraphs, syntax' (corresponding to bytes, packets, flows, and protocol specifications), their difference lies in the fact that each word has precise meanings, while a byte is formed by 8-bit binary digits and hold no inherent meaning. Consequently, we consider using bytes as basic tokens to form an input sequence may lead to ambiguous representations. In this paper, we adopt the bi-gram method to generate tokens, where each token is composed of two adjacent bytes, and the symbol '##' is added to the front of the sequence p_i^m along with the first byte to form the token. As shown in the preprocess part of Fig. 1, the raw hexadecimal sequence p_i^m is transformed into a token sequence S_i^m using the bi-gram method. In addition, we introduce the special tokens [CLS], [MASK], [SEP], [SEG], [UNK], and [PAD] for pre-training. The maximum length L of the token sequence S_i^m is set to 256, with the token [PAD] used for padding shorter sequences.

In this framework, we first preprocess the sequence S_i^m in two separate steps to produce two distinct sequences: the Byte-Flow-Level sequence K_{bf}^{mi} and the Packet-Level sequence K_p^{mi}. This dual-step preprocessing is designed to capture different aspects of the datagram's structure at both the byte-flow and packet levels. Subsequently, these two sequences are concatenated along with the token [SEG] to create a unified input sequence U^{mi}. The inclusion of the [SEG] token facilitates the distinction between the different types of sequences within the model. By integrating data from both sequences in this manner, we aim to mitigate the potential dominance of information from a single pre-training task on the learning process of others.

On the one hand, we add token [CLS] to the beginning of the sequence S_i^m and replace some right tokens with [MASK] or other tokens to form sequence K_{bf}^{mi}. On the other hand, for creating K_p^{mi}, we first divide the hexadecimal sequence p_i^m into a header sequence p_h^{mi} and a payload sequence p_p^{mi}, then generating token sequences S_h^{mi} and S_p^{mi} respectively. In general, the token sequences S_h^{mi} and S_p^{mi} are same-origin packet sequence. We then randomly shuffle the corresponding relationship between the token sequences S_h^{mi} and S_p^{mi}. Using Figure 1. as an example, we keep the relationship between the token sequences S_h^{mi} and S_p^{mi} of $packet0$ unchanged, and we concatenate the token sequence S_h^{mi} of $packet1$ with the token sequence S_p^{mi} of $packet2$, similarly, the token sequence S_h^{mi} of $packet2$ is concatenated with the token sequence S_p^{mi} of $packet1$. Finally, we insert the token [SEP] between the token sequences S_h^{mi} and S_p^{mi} to connect them and replace tokens same to K_{bf}^{mi}, forming the packet-level sequence K_p^{mi}. The preprocess and pre-training stages of Figure 1 provide a detailed demonstration of this process.

B. Pre-Training Tasks

We propose three pre-training tasks to obtain traffic patterns from different perspectives. The first task is aimed at predicting the hidden state features of flows to uncover traffic patterns at the flow level. The second task focuses on acquiring knowledge of protocol paradigms and constraints by predicting the relationship of same-origin packets at the packet level. The third task captures the contextual relationships of traffic by predicting masked tokens at the byte level. The process of the pre-training tasks is shown in the pre-training stage of Fig. 1.

1) Hidden State Feature Prediction: The goal of HSP is to explore pattern knowledge of datagram structure at the flow level. However, the concept of "traffic patterns" is inherently abstract, making it challenging to define with precise numerical values or boundaries. The study [33] highlights that by examining the relationships between state features–which

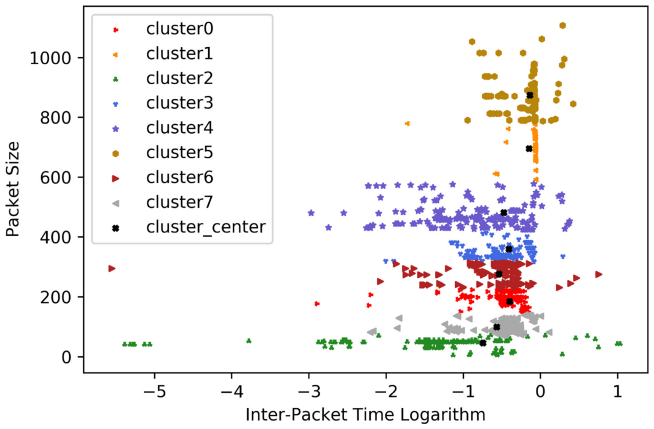


Fig. 3. Hidden state feature of the flows.

encapsulate side-channel data from various flows–one can discern distinctive traffic fingerprinting. Inspired by this notion, we aim to acquire pattern knowledge at the flow level by analyzing the relationship between flows and their respective state features. Given that the side-channel information of flows at different times can constitute different state features, we assume that the average values of these time-varying data may reflect the overall state feature of the flow. Corresponding to the "state feature" proposed in the work [33], we consider the states reflected by the average values as "hidden state feature".

To obtain hidden state features, we utilize side-channel data such as packet size and inter-packet time. These are fundamental packet attributes that dynamically vary in response to alterations in the content being transmitted and changes in the network conditions. We calculate the average packet size and the logarithm of the average inter-packet time for each flow f_i to form observation values. Subsequently, these values are clustered to form hidden state features. The formulas for computing the hidden state feature are as follows:

$$Z_i = \frac{\sum_{j=1}^{M} s\left(p_i^j\right)}{M} \tag{4}$$

$$T_i = \log_2\left(\frac{\sum_{j=1}^{M-1}\left|t(p_i^j) - t(p_i^{j+1})\right|}{M}\right) \tag{5}$$

$$S_i = cluster(Z_i, T_i) \tag{6}$$

where Z_i and T_i represent the average packet size and the logarithm of the average inter-packet interval time of flow f_i respectively, $s(\cdot)$ represents packet size, $t(\cdot)$ is arriving time of a packet, M represents the total number of packets and *cluster* means the K-means method, S_i represents the hidden state feature of f_i.

To validate the assumption of hidden state features, we cluster 2000 flows from the ISCX-VPN dataset using the method according to equation (6), and the result is shown in Fig. 3. The result illustrated that the traffic is divided into 8 categories, which have clear boundaries between them, i.e., the results can be regarded as the hidden state features of eight different flows.

The acquisition of hidden state features can contribute to extracting features at the flow level. Specifically, we train the model to infer the relationship between the input sequence U^{mi} and the hidden state feature h_i of the flow. Setting θ denotes the model parameters, and the loss function of the task is as below:

$$L_{\text{hsp}} = -\sum_{i=1}^{N}\sum_{m=1}^{M} \log P\left(h_i | U^{mi}, \theta\right) \tag{7}$$

2) Same Packet Prediction: The purpose of SPP is to learn knowledge of datagrams by exploring the structure at the packet level. Here, we investigate the relationship between the headers and the payloads. Packets serve as the fundamental units of network transmission, which include sufficient metadata information. Regardless of how privacy-enhanced technologies randomize or standardize packet attribute values, packets can still independently route and be verified. Thus, we consider that this metadata information, which exists within both the header and payload, reflects the organizational paradigm of the packet. On one hand, different protocols are selected and marked in the header based on different transmission content. On the other hand, metadata in the header is directly related to the payload; for example, the length field in the header is calculated by considering the length of the encrypted payload. Conversely, the payload, in turn, affects the checksum for transmission verification. Thus, a reasonable packet should satisfy the mutual relationship/constraint between the header and payload.

To visually demonstrate this constraint, we adopt the method proposed in [17], converting the hexadecimal bytes of headers and payloads into grayscale images to analyze their relationship. Specifically, for each packet, we convert the bytes of the header and payload into distinct matrices. Each byte is mapped to a grayscale value between 0 and 255, mirroring the pixel intensity range in a grayscale image. This process enables the transformation from packets to corresponding grayscale images, as depicted in Fig. 4. For each type of packet, the grayscale image of the header is displayed at the top, and the payload is displayed at the bottom. Upon comparing these images, we notice subtle differences in the grayscale patterns of payloads corresponding to the same header type, evident in Fig. 4(11) and (15), as well as Fig. 4(12) and (16). In contrast, even within the same packet type, significant variations are observed in the payload grayscale images of different header types, as demonstrated in Fig. 4(2) and (3), and Fig. 4(7) and (8).

Therefore, we consider that there is a certain pattern between the header and payload that determines the validity of the packet. Given a packet header, the payload is allowed to change within a certain range rather than randomly. This pretraining task aims to discover this pattern, learn the constraint rules between packet header and payload. Specifically, when constructing K_p^{mi}, we randomly shuffle the header and payload from different packets, where 50% of the relationships are correct, 30% of the payloads are replaced with payloads from other packets of the same flow, and the remaining 20% of the payloads are replaced with payloads from packets of different

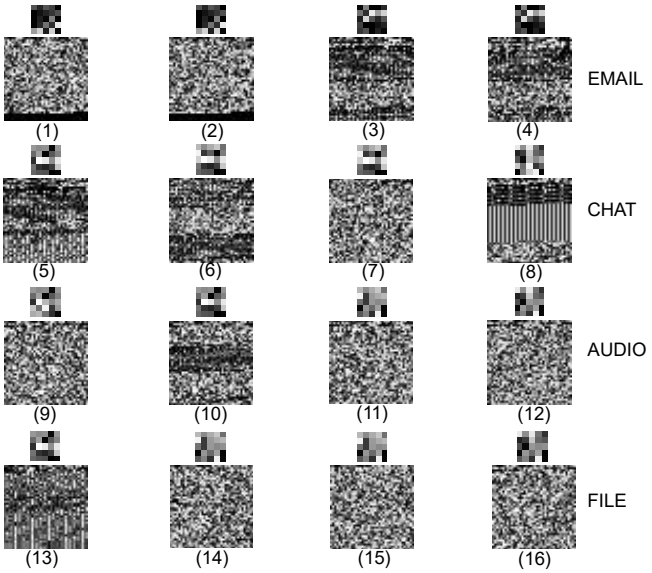


Fig. 4. Grayscale images of packet headers and payloads for different types of packets.

flows. We train the model to predict whether the relationship between the header and payload is of the same origin, where the relationship is denoted as $y_i^m \in [0, 1]$, with 0 indicating that the payload and header come from the same packet, and 1 indicating the opposite. The loss function for this task is as below:

$$L_{spp} = -\sum_{i=1}^{N}\sum_{m=1}^{M} \log P(y_i^m | K_p^{mi}, \theta) \tag{8}$$

3) Masked Token Prediction: MTP aims to explore the knowledge of datagram structure at the byte level. As mentioned previously, flow can be regarded as a language between different applications in the network, and similar to natural language, flow exhibits structure and specification. Therefore, we adopt the Masked Language Model to construct the pre-training task for discovering contextual relationships of traffic. Specifically, in the pre-training process, each token in the input sequence is randomly masked with a 20% probability. For each selected token, we replace it with the special token [MASK] with a probability of 80%, or choose another token (excluding the special tokens) to replace it with a probability of 10%, or leave it unchanged with a probability of 10%. The pre-training model is trained to predict the masked tokens based on context. In the implementation, we first design a dictionary containing all tokens involved in the pre-training process and assign them numerical indices. Then, when replacing a token, we simultaneously create a tuple containing the position and the index of the replaced token. During pre-training, this tuple aids the model in predicting the token index. The loss function for this task is as follows:

$$L_{mtp} = -\sum_{i=1}^{N}\sum_{m=1}^{M} \log P(mask_j^{mi} = token_j^{mi} | K_{bf}^{mi}, \theta) \tag{9}$$

TABLE III
THE DETAILS OF DATASETS USED IN MULTI-TASK COMPARATIVE EXPERIMENTS

Task	Dataset	#flow	#packet	#label
Encapsulation detection	ISCX-VPN	5680	62982	12
Anonumous detection	ISCX-Tor	2581	64756	14
Aplication detection	CSTNET-TLS1.3	46372	581709	120
Malware detection	USTC-TFC	10945	154320	20
Attack detection	ToN-IoT	4352	58512	9

The final pre-training objective is defined as the sum of the aforementioned three losses:

$$L_{ALL} = L_{hsp} + L_{spp} + L_{mtp} \tag{10}$$

C. Finetune

After pre-training, the model can generate generic and robust representations for general encrypted traffic. However, in specific traffic identification tasks, the representations cannot fully capture the particular task details, and fine-tuning is necessary for more accurate results. The fine-tuning stage involves a different model structure compared to the pre-training stage. In the pre-training stage, the model adds three additional networks after the transformer encoder layer to complete the pre-training tasks and adjust the parameters of the transformer to generate representations. However, these three networks are removed in the fine-tuning stage, and a linear layer is added to execute specific identification. The first token [CLS] of the representation can be directly used for classification, as shown in the fine-tuning stage of Fig. 1.

IV. EXPERIMENT AND ANALYSIS

In this section, we primarily focus on evaluating the performance of the proposed framework across different tasks as well as assessing its robustness in unbalanced scenarios and traffic defense mechanism scenarios. To achieve this, we design experiments, including multi-task comparative experiments, imbalanced encrypted traffic identification experiments, and traffic defense mechanism resistance experiments. To illustrate the necessity of the three pre-training tasks, we conduct ablation experiments. Finally, we present the evaluation of complexity.

A. Experimental Setup

1) Dataset and Tasks: We conduct experiments on different datasets, including ISCX-VPN, ISCX-Tor [34], ToN-IoT [35], USTC-TFC [36], and CSTNENT-TLS1.3 [12], with each dataset is associated with a specific task, and the detailed information of the datasets is shown in Table III.

ISCX-VPN: The dataset includes popular VPN protocols such as OpenVPN and OpenConnect. Considering encapsulation and service types of the traffic [17], we categorize this dataset into 12 labels: Email, Chat, Streaming, Transfer, VoIP,

P2P, VPN-Email, VPN-Chat, VPN-Streaming, VPN-Transfer, VPN-VoIP, and VPN-P2P. We use this dataset to assess the approach's performance in encapsulation detection tasks.

ISCX-Tor: The dataset includes a large volume of traffic captured from the Tor network, including Web browsers, Email clients, and P2P file-sharing applications. Following the setting in [12], we categorize this dataset into 14 labels: Browsing, Chat, Email, Transfer, P2P, Streaming, VoIP, Tor-Browsing, Tor-Chat, Tor-Email, Tor-Transfer, Tor-P2P, Tor-Streaming, and Tor-VoIP. We utilize this dataset to validate the method's ability to perform anonymity detection tasks.

USTC-TFC: The dataset includes encrypted traffic from malware such as Tinba, Miuref, and Cridex, as well as benign traffic from applications like Facetime and Gmail. We use this dataset to test the method's effectiveness in malware detection tasks.

ToN-IoT: The dataset was collected from a realistic and large-scale network designed at the Cyber Range and IoT Labs of SEIT and UNSW [37]. Here, we utilize the network dataset for experiments, which includes nine types of attacks: backdoor, DDoS, DoS, Injection, MITM, Password, runsomware, XSS, and scanning.

CSTNET-TLS1.3: The dataset consists of traffic from 120 applications, encrypted with TLS 1.3, collected by the University of Science and Technology of China.

2) Evaluation Metrics: We evaluate and compare the performance of the methods using four typical metrics, including accuracy (ACC), precision (PRE), recall (REC), and F1 score, and the calculation methods of these metrics are as follows:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

$$PRE = \frac{TP}{TP + FP} \tag{12}$$

$$REC = \frac{TP}{TP + FN} \tag{13}$$

$$F1 = 2 * \frac{PRE * REC}{PRE + REC} \tag{14}$$

3) Implementation: We remove the IP and port information because they are irrelevant to the specific traffic of the transmitted content and do not contribute to generating the representation. All experiments were conducted on the Ubuntu system using PyTorch 1.8.0 and NVIDIA RTX 2080ti. In the pre-training phase, we set the embedding size $=$ 512, hidden size $=$ 512, learning rate $= 5 \times 10^{-4}$, batch size $=$ 50, and train the model for 8×10^5 steps. The hyperparameter search ranges and the selected values are shown in Table IV. We utilize 25GB of public dataset data (20GB from ToN-IoT, 5GB from ISCX-VPN) to generate a total of 50GB of raw corpus for pre-training. For each pcap file processed after the 5-tuple method, we use all packets in the pcap file to construct K_p^{mi} according to the method mentioned in Section III-B2. Since K_p^{mi} and K_{bf}^{mi} involve different processes for the same metadata, we also utilize all hexadecimal bytes of the same packet to make the token sequence (retaining only the 256 tokens). In the fine-tuning phase, for each flow, we only extract one packet to generate the token sequence for identification.

TABLE IV
HYPERPARAMETERS SELECTION FOR GETRF

Hyperparameters	Search Range	Final
Number of Transformer Encoders	[10 ,..., 20]	12
Number of Embedding Size	[128 , 256 , 512]	512
Number of Hidden Units	[128 , 256 , 512]	512
Learning rage	[0.0001 ,..., 0.005]	0.0005
Pre-training Steps	$[5 \times 10^5$,..., $8 \times 10^5]$	8×10^5
Dropout	[0,...,0.5]	0.1
Activation Functions	[Tanh, Relu, Elu]	Tanh
Batch Size	[30 ,..., 50]	50
Optimizer	[Adam, RMSProp, SGD]	Adam

We set the proportions of the training, validation, and testing sets to 1:1:2, and evaluate the framework using the ten-fold cross-validation method.

B. Multi-Task Comparative Experiment

In these experiments, we separately evaluate the performance of the proposed framework on five different tasks: malware detection, application classification, anonymous detection, attack detection, and encapsulation detection. Moreover, we compare GETRF with various methods, including DF [18], DeepPacket [20], LeNet [38], Var-CNN [19], AppScanner [15], App-Net [26], BIND [16], CUMUL [14], FlowPrint [40], GraphDApp [22], FS-Net [39], GCN_Transformer [21], and ET-BERT [12]. For details on the implementation of the baseline models, please refer to Appendix C.

Table V presents the performance of different methods in encapsulation detection, anonymous detection, and application classification. GETRF performs best across all four metrics in these three tasks. Compared to the current state-of-the-art method, GETRF achieves an average improvement of 0.512% in accuracy, 0.448% in precision, 0.488% in recall, and 0.464% in F1 score across the three tasks. In the application classification task, which has the most categories, GETRF outperforms ET-BERT by 0.78% in accuracy and 0.76% in F1 score. One possible reason for this improvement is that the CSTNETE-TLS.13 dataset contains 120 different categories, with many similar traffic patterns. GETRF mitigates the impact of single patterns on the results by learning datagram structure knowledge across multiple levels and achieves fine-grained application identification.

Table VI presents the performance of different methods in malware detection and attack detection. Our model achieves the best performance on F1 score at 99.85% and 98.80% respectively. We notice that some traffic in USTC-TFC includes unencrypted data at the Application Layer. This aspect simplifies the task for other models, as they can readily utilize this plaintext for easier classification.

Overall, GETRF achieves the best performance in all five tasks. Additionally, we find that representation-based methods outperform other methods, indicating that pre-training can help models learn faster and is suitable for identifying large-scale flow data with a small number of sample data.

TABLE V
COMPARISON RESULT ON ENCAPSULATION DETECTION, ANONYMOUS DETECTION, AND APPLICATION CLASSIFICATION TASKS

Task	Encapsulation detection				Anonymous detection				Application classification			
Dataset	ISCX-VPN				ISCX-Tor				CSTNET-TLS1.3			
Method	ACC	PRE	REC	F1	ACC	PRE	REC	F1	ACC	PRE	REC	F1
DF	0.7589	0.7435	0.7232	0.7332	0.7832	0.7588	0.7543	0.7565	0.7381	0.7039	0.7195	0.7120
Deeppacket	0.8397	0.7999	0.8074	0.8036	0.7588	0.7278	0.7029	0.7151	0.7547	0.3939	0.2218	0.2836
LeNet	0.8055	0.7193	0.6522	0.6841	0.3465	0.3329	0.3149	0.3236	0.6595	0.6333	0.6772	0.6544
Var-CNN	0.8644	0.8328	0.8248	0.8287	0.7130	0.7018	0.6932	0.6932	0.7060	0.7041	0.7149	0.7093
AppScanner	0.7042	0.7129	0.6937	0.7031	0.6633	0.4890	0.4125	0.4475	0.6841	0.6421	0.6358	0.6391
BIND	0.7324	0.7532	0.7189	0.7356	0.7549	0.5982	0.6679	0.6311	0.7962	0.7509	0.7651	0.7579
CMUML	0.5560	0.5638	0.5473	0.5554	0.4498	0.3725	0.4423	0.4044	0.5170	0.4831	0.4971	0.4900
FlowPrint	0.8161	0.7927	0.7732	0.7828	0.8523	0.3922	0.3561	0.3732	0.1135	0.1311	0.1128	0.1212
APP-Net	0.7636	0.5560	0.6343	0.5925	0.2644	0.1714	0.1889	0.1829	0.7875	0.7345	0.7113	0.7227
GraphDApp	0.6541	0.5821	0.5710	0.5764	0.6530	0.4832	0.4483	0.4650	0.7174	0.6562	0.6514	0.6547
FS-Net	0.7420	0.5821	0.6640	0.6023	0.5928	0.5932	0.6022	0.5976	0.7558	0.7303	0.7096	0.7197
GCN_Transformer	0.7988	0.7642	0.7527	0.7584	0.7829	0.7623	0.7772	0.7697	0.8547	0.8797	0.8685	0.8769
ET-BERT	0.9890	0.9891	0.9890	0.9890	0.9922	0.9930	0.9930	0.9930	0.9737	0.9742	0.9742	0.9741
GETRF	**0.9936**	**0.9925**	**0.9910**	**0.9916**	**0.9975**	**0.9940**	**0.9984**	**0.9961**	**0.9815**	**0.9816**	**0.9820**	**0.9817**

TABLE VI
COMPARISON RESULT ON MALWARE DETECTION AND ATTACK DETECTION TASKS

Task	Malware detection				Attack detection			
Dataset	USTC-TFC				ToN-IoT			
Method	ACC	PRE	REC	F1	ACC	PRE	REC	F1
DF	0.7188	0.7325	0.7159	0.7241	0.6542	0.6133	0.5973	0.6051
Deeppacket	0.9045	0.8955	0.9047	0.9001	0.7557	0.7552	0.7578	0.7564
LeNet	0.8566	0.5644	0.4425	0.4961	0.5525	0.3966	0.3946	0.3955
Var-CNN	0.8577	0.8329	0.8436	0.8382	0.6012	0.5578	0.5744	0.5659
AppScanner	0.8737	0.8526	0.8633	0.8597	0.7885	0.7790	0.7845	0.7817
BIND	0.8999	0.8520	0.8869	0.8691	0.7665	0.7438	0.7380	0.7408
CMUML	0.5858	0.5911	0.5721	0.5814	0.5855	0.5236	0.5371	0.5302
FlowPrint	0.7923	0.5923	0.6356	0.6136	0.8032	0.7113	0.7182	0.7147
APP-Net	0.9266	0.9266	0.8953	0.9060	0.8351	0.7921	0.7562	0.7964
GraphDApp	0.8933	0.8132	0.8253	0.8537	0.8540	0.8426	0.8460	0.8442
FS-Net	0.8845	0.8724	0.8629	0.8784	0.5478	0.5042	0.5209	0.5125
GCN_Transformer	0.8533	0.8289	0.8094	0.8190	0.8185	0.8117	0.8044	0.8081
ET-BERT	0.9967	0.9964	0.9964	0.9964	0.9853	0.9812	0.9794	0.9802
GETRF	**0.9987**	**0.9985**	**0.9986**	**0.9985**	**0.9912**	**0.9897**	**0.9864**	**0.9880**

C. Imbalanced Encrypted Traffic Identification Experiment

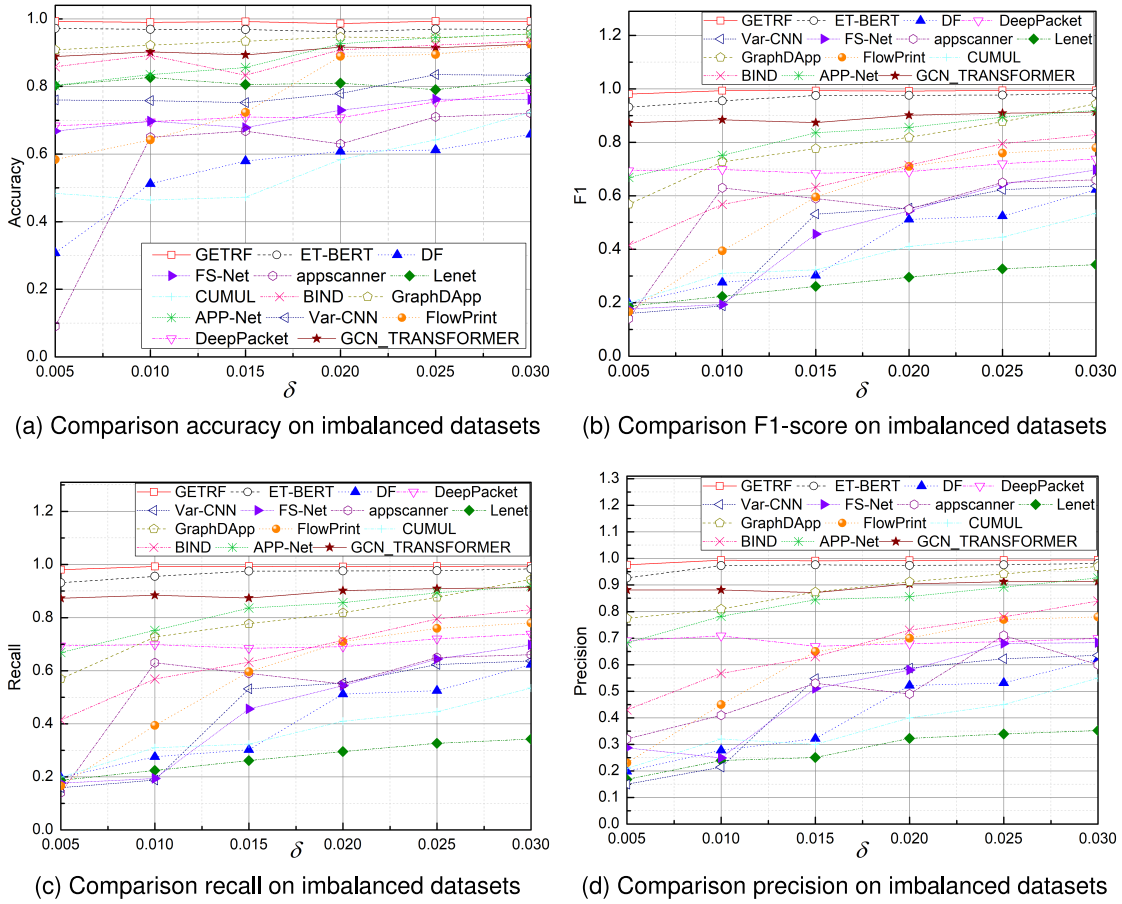
Generally, datasets for specific tasks typically include only relevant protocols and applications, without other traffic interference. However, in real-world environments, a large amount of background traffic exists, and its volume can be 100 times or more than that of the task-relevant traffic [41]. Therefore, we design experiments to verify the fine-grained identification performance of GETRF.

In these experiments, we select traffic from Facebook, Netflix, ICQ, and Vimeo in the ISCX-VPN dataset as background traffic, and traffic from Virut, NSIS, Geodo, Cridex, Miuref, Password, Backdoor, XSS, and DDoS in USTC-TFC and ToN-IoT as the target malicious flows. We also set δ as the threshold to control the volume of target malicious flows. For example, $\delta = 0.005$ means that the maximum number for each type of target malicious flow is 0.005 times that of any background traffic. The specific details of the dataset are shown in Table VII.

TABLE VII
THE DETAILS OF DATASETS USED IN IMBALANCED ENCRYPTED TRAFFIC IDENTIFICATION EXPERIMENTS

quantity	#packet					
θ	0.005	0.01	0.015	0.02	0.025	0.03
Facebook	7804	7782	7800	7800	7802	7798
Virut	20	44	84	96	124	215
Nsis	20	58	72	93	114	190
password	32	52	72	92	176	207
backdoor	31	56	102	132	120	204
XSS	36	44	70	100	123	199
Cridex	30	64	82	92	154	225
Miuref	20	48	76	108	122	235
Geodo	14	40	78	100	128	159
ICQ	7324	7282	7359	7320	7320	7406
Netflix	8546	8486	8523	8474	8446	8382
Vimeo	7868	7890	7502	7838	198	7878
DDoS	32	52	102	127	186	215

As shown in Fig. 5, most methods show improved performance as δ increases, especially in terms of precision and recall. This improvement indicates that the methods



(a) Comparison accuracy on imbalanced datasets

(b) Comparison F1-score on imbalanced datasets

(c) Comparison recall on imbalanced datasets

(d) Comparison precision on imbalanced datasets

Fig. 5. Comparison of the results of different methods under various δ scenarios.

learn more about the target flow from the labeled data during training, thus enhancing their performance. However, a model with good generalization capabilities needs to maintain high performance even under conditions of limited samples. Fig. 5 (c) and (d) demonstrate that pre-training designs like GETRF and ET-BERT outperform other models. Particularly under the extreme condition of $\delta = 0.005$, our method surpasses ET-BERT, with precision and recall improvements of 4.96% and 4.85%, reaching 97.62% and 98.59% respectively. As δ increases, the gap narrows, indicating that imbalanced data affects model performance and highlighting our method's effectiveness in generalizing representations of encrypted traffic.

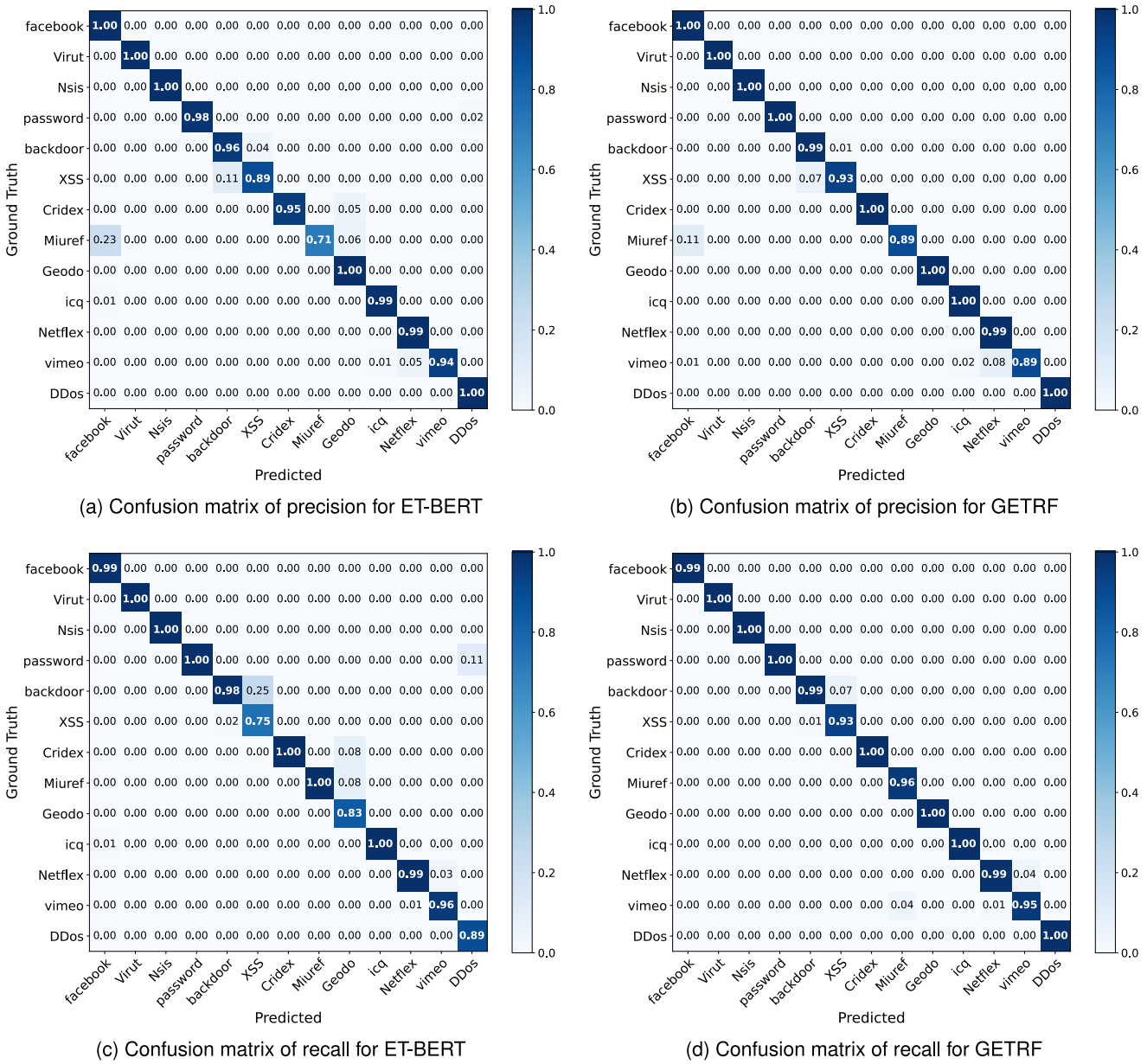
Fig. 6 presents the precision and recall confusion matrices for GETRF and ET-BERT with δ=0.005. In this scenario, the proportion of malicious flows is very small compared to the background traffic, limiting the prior knowledge that the methods can acquire. The results show that GETRF outperforms ET-BERT in both precision and recall. GETRF improves the recall score of XSS and Geodo from 75% to 93% and from 83% to 100%, respectively, and the precision score of Miuref from 71% to 89%. These findings indicate that our framework effectively learns the characteristics of traffic for identification, mitigating the impact of background traffic to achieve fine-grained classification with a small number of target samples.

D. Traffic Defense Mechanism Resistance Experiment

In this experiment, we adopt Front [29] to disturb the traffic pattern. To directly demonstrate the impact of Front on traffic patterns, we apply WeFDE [42] to measure the amount of information leakage of traffic features before and after the application of Front.

Firstly, we sample data from ToN-IoT and label it as D_1, then we make the dataset as described in IV-C under the setting with δ equal to 0.005, and label it as D_2. Fig. 7 shows the information leakage of the features in both datasets before and after being processed by Front. Clearly, Front successfully reduces the information leakage of the features, with an average reduction of 0.3956 bit and 0.5842 bit for 60 features in D_1 and D_2 respectively. On D_1 and D_2, the total amount of information provided by 60 features after Front interference accounts for 27.57% and 25.58% of the original, respectively. For details about the calculation of information leakage and the features selected for these experiments, please refer to Appendix A.

Fig. 8 presents the performance of GETRF in D_1 and D_2. It can be observed that the accuracy, precision, recall, and F1 score of the proposed framework have decreased by 4.80%, 4.89%, 4.40%, and 4.65% on D_1, and by 0.83%, 2.13%, 4.8%, and 3.48% on D_2, respectively. In comparison, the total amount of effective information decreased by 72.43% in D_1 and by 74.42% in D_2. Additionally, GETRF still performs



(a) Confusion matrix of precision for ET-BERT

(b) Confusion matrix of precision for GETRF

(c) Confusion matrix of recall for ET-BERT

(d) Confusion matrix of recall for GETRF

Fig. 6. Confusion matrices of traffic identification for GETRF and ET-BERT ($\delta = 0.005$).

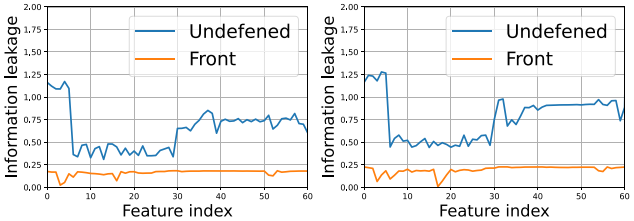


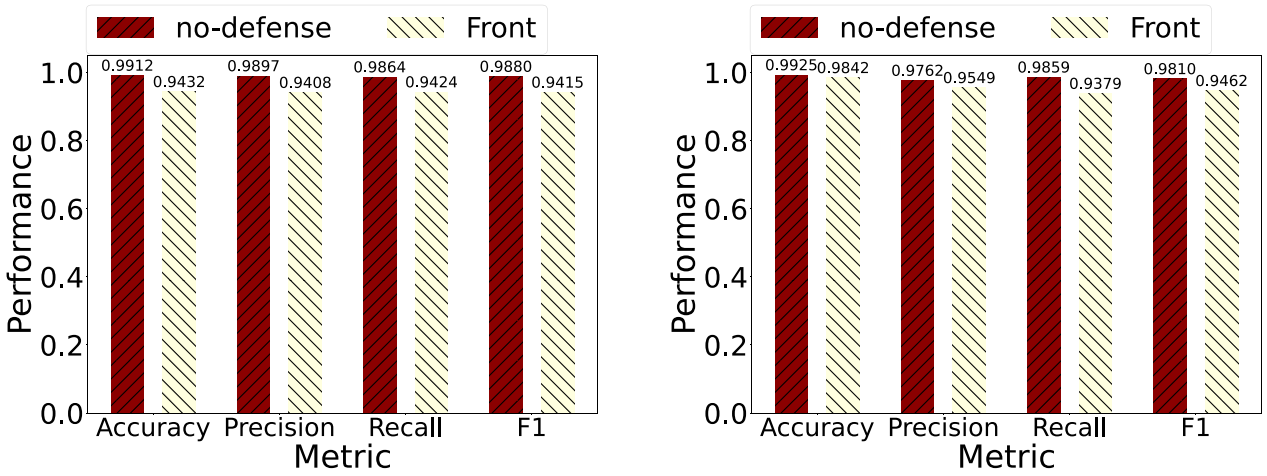
Fig. 7. Information leakage for individual feature on D1(left) and D2(right).

better than most methods in accuracy according to IV-B and IV-C under these conditions.

We consider that the performance of GETRF may be attributed to two factors. For a given traffic, GETRF only utilizes the first few bytes to generate a 256-token sequence for identification, rather than using all metadata in the flow, which may mitigate some impacts of Front. Secondly, although Front can potentially affect feature distribution and lead to the misclassification of the model, GETRF includes pre-trained knowledge at both the flow and packet levels. This allows it to analyze the correlation among protocols, payload, and burst features of dummy packets, thereby reducing the impact.

E. Ablation Study

We present ablation results to validate each component's contribution to our method's performance on the ToN-IoT dataset. We first evaluate the method's performance when the specific pre-training task is omitted. As shown in Table IX, compared to the complete framework, the F1 scores decrease by 7.56%, 14.45%, and 26.52%, respectively. This indicates



(a) Performance of GETRF on D1

(b) Performance of GETRF on D2

Fig. 8. Performance of GETRF under traffic defense mechanism.

TABLE VIII
COMPARISON RESULT OF COMPLEXITY

Stage	Methods	parameters	Training Time	Interfence Time	Overall Time	Performance (F1 score)				
						AC	AD	MD	ND	ED
Preparation	ET-BERT	546MB	1.69	-	-	-	-	-	-	-
	GETRF	343MB	**1**	-	-	-	-	-	-	-
Deployment	Var-CNN	16MB	0.11	0.06	0.17	0.7093	0.5659	0.8382	0.6932	0.8287
	DF	12MB	0.08	0.04	0.12	0.7120	0.6051	0.7241	0.7565	0.7332
	APP-Net	10MB	0.12	0.05	0.17	0.7227	0.7964	0.9060	0.1829	0.5925
	GCN_Transformer	144MB	0.84	0.3	1.14	0.8769	0.8081	0.8190	0.7697	0.7584
	ET-BERT	326MB	1.35	0.45	1.8	0.9741	0.9802	0.9964	0.9930	0.9890
	GETRF	201MB	**1**	0.37	1.37	0.9817	0.9880	0.9985	0.9961	0.9916

[1]In the Performance columns, AC is Application Classification, AD is Attack Detection, ND is Anonymous Detection, ED is Encapsulation Detection, MD is Malware Detection.

that the three pre-training tasks are complementary and provide different levels of information for accurate traffic representation. Notably, the framework without MTP performs the worst, indicating that learning the traffic patterns at the byte-level is crucial for traffic representation. This also supports the view presented in [43].

Additionally, to further analyze the impact of different pre-training tasks on the final traffic representations, in Appendix B, we generate the traffic representations using the MTP, SPP, and HSP tasks respectively, and then visualize the results.

We also observe that the model without pre-training has a significant drop in precision, recall, and F1 score by 37.72%, 37.76%, and 37.74%, respectively. This demonstrates that although transformer models have some advantages in capturing flow features, pre-training can further improve the model's accuracy.

F. Evaluation of Complexity

In this section, we analyze the complexity of GETRF, including trainable parameters and overall time costs. We choose ET-BERT, GCN_Transformer, APP-Net, DF, and Var-CNN as comparative objects because their performance in Section IV-B closely matches GETRF's. Additionally, these methods involve architectures such as CNN, LSTM, Transformer, and GCNs, which can provide a more comprehensive analysis.

As shown in Table VIII, we present the parameters, training time, inference time, and performance of the methods. The preparation stage refers to knowledge extraction before the model is applied to specific tasks, while the deployment stage means training and inference based on labeled data. Note that the training time cost for GETRF is set to 1, serving as the time cost baseline for other methods.

From Table VIII, it can be observed that in the deployment stage, the parameters and time costs for Var-CNN, DF, and APP-Net are less than those of other methods. However, these three methods only exhibit high performance on a specific task, which may be attributed to an insufficient number of training samples and feature extractors. GCN_Transformer, ET-BERT, and GETRF all contain multiple transformer encoders, resulting in a larger number of parameters and more comprehensive feature extraction capabilities. Compared to ET-BERT, GETRF

TABLE IX
ABLATION STUDY OF GETRF

Method	ACC	PRE	REC	F1
w/o spp	0.9175	0.9079	0.9170	0.9124
w/o hsp	0.8778	0.8432	0.8440	0.8435
w/o mtp	0.7492	0.7433	0.7035	0.7228
w/o pre-training	0.6244	0.6125	0.6088	0.6106
full model	**0.9912**	**0.9897**	**0.9864**	**0.9880**

has fewer parameters and consumes less time. This is primarily because the embedding sizes and the number of hidden units of ET-BERT are both 768, leading to a larger space occupation for each traffic mapping vector and increased computational operations. However, in terms of performance, GETRF outperforms ET-BERT, further demonstrating the effectiveness of the three proposed pre-training tasks.

V. CONCLUSION

In this paper, we proposed and implemented GETRF, a novel framework for classifying encrypted network traffic. This approach learns implicit patterns of the datagram structure at three distinct levels from large-scale unlabeled traffic data, thereby generating generic and robust traffic representations. GETRF demonstrated superiority in multi-class identification of encrypted network traffic. Additionally, in two independent studies, we evaluated the performance of GETRF in handling imbalanced data and traffic with disturbed features. In terms of overall accuracy and F1 score metrics, the results show that GETRF outperforms other reference methods. In future work, we plan to collect a larger dataset encompassing a wider range of protocols, features, and service types to further validate our model. We aim to explore useful and meaningful insights into the datagram structure and investigate metrics for a comprehensive assessment of our approach in fine-grained identification scenarios.

ACKNOWLEDGMENT

The authors are very grateful to Professor Mei Song for her guidance and help with this article. They also thank Professor Yinglei Teng for her invaluable guidance and insights throughout the development of this paper. Finally, they thank the anonymous reviewers for their constructive comments, corrections, and inspiration to improve this paper.

APPENDIX A
INFORMATION LEAKAGE MEASUREMENT

In this section, we discuss the theoretical framework for calculating information leakage from traffic features, and the selection of features for our experiments.

The information leakage of a feature can be calculated using mutual information, and the specific calculation method is as follows.

$$I(C;v) = H(C) - H(C|v) \tag{15}$$

$$H(C|v) = \int_{\phi} p(x)H(C|x)dx$$

TABLE X
FEATURE DESCRIPTION

Index	Description
1	The total packet count
2-3	The count of outgoing and incoming packets
4-5	The ratio of incoming packets and outgoing packets to the total packet count
6-8	The maximum, average, and standard deviation for all packet inter-arrival time
9-12	The maximum, average, standard deviation, and 3rd quartile for all incomming packet time
13-16	The maximum, average, standard deviation, and 3rd quartile for all outgoing packet time
17	The total transmission time of all packets
18-20	The 1st, 2nd (median), and 3rd quartile transmission times of all packets
21	The total transmission time of all incomming packets
22-24	The 1st, 2nd (median), and 3rd quartile transmission times of all incomming packets
25	The total transmission time of all outgoing packets
26-28	The 1st, 2nd (median), and 3rd quartile transmission times of all outgoing packets
29	The average packet size
30-31	The maximum and average number of bursts in outing packets
32-33	The maximum and average number of bursts in incomming packets
34-36	The number of bursts in outgoing packets where there are at least 5, 10, and 20 packets, respectively.
37-39	The number of bursts in incoming packets where there are at least 5, 10, and 20 packets, respectively.
40-49	Partition the packet sequence into non-overlapping groups of 10 packets each and calculate the number of outgoing packets in the first 10 groups
50-59	Partition the packet sequence into non-overlapping groups of 10 packets each and calculate the number of incoming packets in the first 10 groups
60	The maximum burst count

$$= -\int_{\phi} \sum_{c_i \in C} p(x)pr(c_i|x)\log_2 pr(c_i|x)dx \tag{16}$$

$$H(C) = -\sum_{c_i \in C} pr(c_i)\log_2 pr(c_i) \tag{17}$$

where, $I(\cdot)$represents mutual information, $H(\cdot)$ represents entropy, C represents the category that traffic f belonging to, $pr(c_i)$ represents the probability of c_i belonging to C, ϕ is the domain of feature v, x refers to specific value of v, and $p(\cdot)$ represents the probability density function.

As for features selection, we considered side-channel information related to the traffic, including packet count features, time statistics, packet ordering features, and burst features. With specific descriptions as shown in Table X

APPENDIX B
VISUALIZATION OF TRAFFIC REPRESENTATIONS

Here, we separately pre-train models based on the MTP, SPP, and HSP tasks. During the evaluation phase, we remove the last softmax layer of the framework and obtain the final representation vectors. We utilize the t-distributed stochastic neighbor embedding approach to reduce the dimensionality of these vectors and plot them as two-dimensional images.

In Fig. 9 (b), (c), and (d) represent the traffic representations generated based on the MTP, SPP, and HSP tasks respectively.

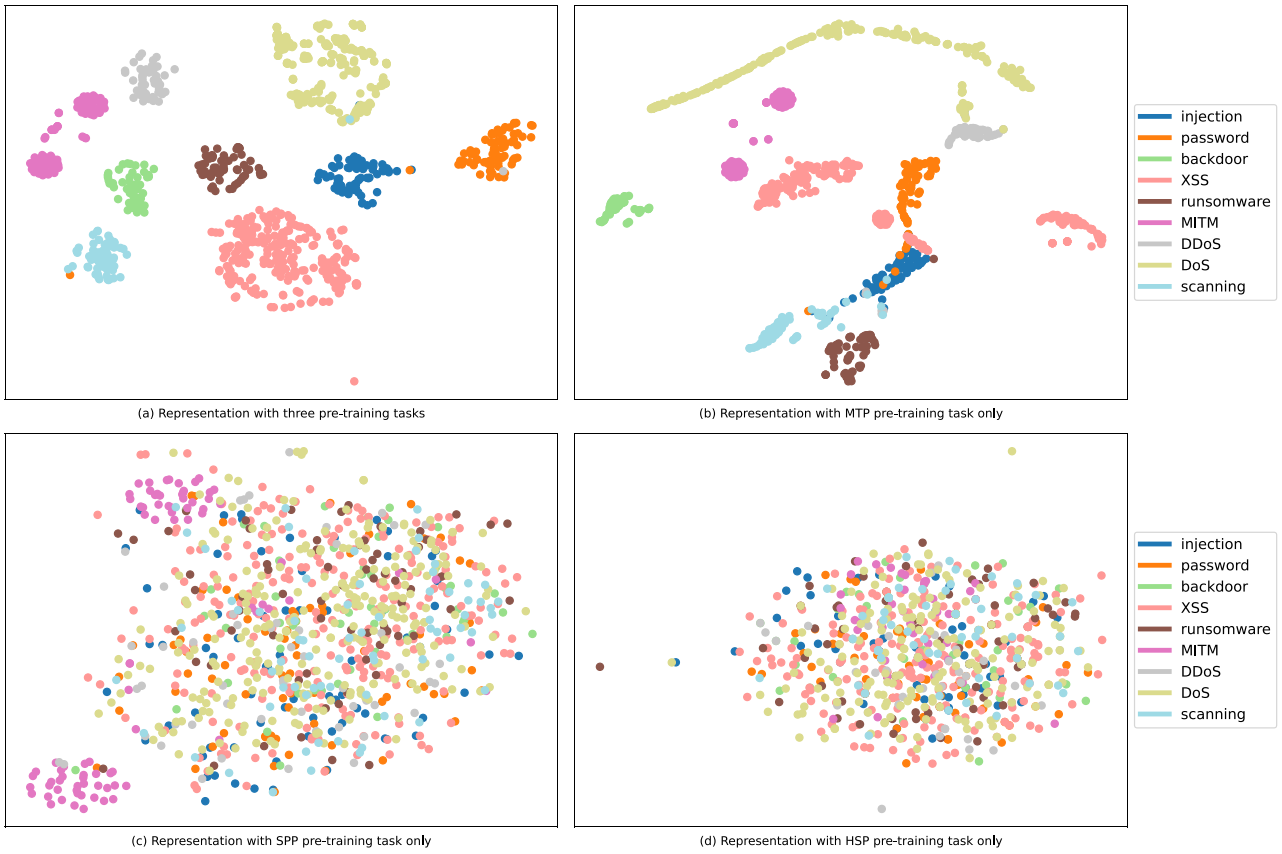


Fig. 9. Visualization of traffic representations with different pre-training tasks on the ToN-IoT dataset.

It can be observed that their performance is less than satisfactory. Among them, (b) performs the best. However, (b) shows disappointing results in classifying password, scanning, and injection traffic. One possible reason is that these flows exhibit behaviors such as multiple login attempts and a large number of connection attempts, showing consistency at the byte level, making them challenging to distinguish. So, additional information is needed to assist in the task.

The traffic representations in (d) are distributed into several different clusters, which may related to the previously mentioned hidden state features. Specifically, the model learns flow behavior from side-channel data. However, different types of flows may exhibit similar burst patterns during transmission, which can lead to similarities in the observed side-channel data. Consequently, the model groups different classes into one cluster.

In summary, the traffic representations generated based on individual pre-training tasks are not satisfactory, and only the representations produced by the complete framework can achieve optimal classification performance, as depicted in Fig. 9 (a).

APPENDIX C
IMPLEMENTATION OF THE BASELINE MODELS

In this section, we discuss the implementation details of the baseline models and the data preparation.

appscanner: This method uses a random forest as the classifier with the threshold set to 0.9, and each decision tree considers a maximum of 6 features during splitting. We utilize statistical features to train the model, including 26 features that score over 2% according to the work [15].

deeppacket: We extract the first 1500 bytes of the packet and replace the IP address with 0, then divide all bytes by 255. We adopt a CNN as the classifier, which is available in [44].

CUMUL: This method uses an SVM as the classifier with an RBF kernel. For input features, we extract the number of incoming and outgoing packets, the total size of incoming packets, and the total size of outgoing packets from pcap files. Then we use 3 as the interval point to obtain equidistant values for 10 packets separately, resulting in 44 feature representations for training.

BIND: This approach uses an SVM as the classifier with an RBF kernel. For input data, we extract packet size, uni-burst size, uni-burst time, uni-burst count, bi-burst size, and bi-burst time for training.

Lenet: We take the first 784 bytes of the first packet from pcap files and convert them into a 28x28 grayscale image to input into the Lenet model for training.

DF: We extract packet direction sequences from each pcap file and store them as pickle files according to the format requirements in [18], then we use the publicly available model [45] for training.

Var-CNN: We extract packet timestamp sequences and packet directions from pcap files and store them according to the requirements in work [46]. Then we train with the available model [47].

FS-Net: We extract packet length sequences from pcap files and train with the available model [39].

APP-Net: We extract sequences of packet lengths and payload bytes from pcap files. Since no source code is available, we implement the model according to the description in the paper [26]. The model consists of two embedding layers, two BiLSTM layers, and two 1D-CNN layers. We set the hidden size of the embedding to 10000, convolutional kernels to 256, the filter size to 7, and the hidden size of BiLSTM to 128.

GraphDapp: We extract the first 10 packets from each pcap file to build a TIG representation and corresponding adjacency matrix. For the model, we construct 3 MLP layers, each containing linear, batch norm, and dropout functions, with 64 hidden units in the linear function and a 25% dropout in the dropout function.

Flowprint: We extract features from pcap files excluding TLS certificates as mentioned in the work [40], and train using the publicly available model [48].

GCN_Transformer: We extract the first 500 bytes from pcap files and construct an adjacency matrix for the text semantic graph based on word co-occurrence frequency and document-word frequency. Point mutual information is used to represent the association between words. The model includes 8 transformer encoders with hidden units set to 512.

ET-BERT: We extract individual packets from flow and adopt the bi-gram method to generate tokens. Then, we build a dictionary to map the tokens into sequences and train using the publicly available model [12].

REFERENCES

[1] M. He, X. Wang, P. Wei, L. Yang, Y. Teng, and R. Lyu, "Reinforcement learning meets network intrusion detection: A transferable and adaptable framework for anomaly behavior identification," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 2477–2492, Apr. 2024, doi: 10.1109/TNSM.2024.3352586.

[2] M. He, Y. Huang, X. Wang, P. Wei, and X. Wang, "A lightweight and efficient IoT intrusion detection method based on feature grouping," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 2935–2949, Jan. 2024, doi: 10.1109/JIOT.2023.3294259.

[3] B. Wang, J. Zhang, Z. Zhang, L. Pan, Y. Xiang, and D. Xia, "Noise-resistant statistical traffic classification," *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 454–466, Dec. 2019.

[4] M. D. Moizuddin and M. V. Jose, "A bio-inspired hybrid deep learning model for network intrusion detection," *Knowl. Based Syst.*, vol. 238, Feb. 2022. Art. no. 107894.

[5] H. Yan, L. Fu, Y. Qi, L. Cheng, Q. Ye, and D. J. Yu, "Learning a robust classifier for short-term traffic state prediction," *Knowl. Based Syst.*, vol. 242, Apr. 2022, Art. no. 108368.

[6] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.

[7] E. Papadogiannaki and S. Ioannidis, "A survey on encrypted network traffic analysis applications, techniques, and countermeasures," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–35, Jul. 2022.

[8] M. Macas, C. Wu, and W. Fuertes, "A survey on deep learning for cybersecurity: Progress, challenges, and opportunities," *Comput. Netw.*, vol. 212, Jul. 2022, Art. no. 109032.

[9] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowl. Based Syst.*, vol. 189, Feb. 2020, Art. no. 105124.

[10] C. Wang et al., "Addressing the train-test gap on traffic classification combined subflow model with ensemble learning," *Knowl. Based Syst.*, vol. 204, Sep. 2020, Art. no. 106192.

[11] H. Y. He, Z. G. Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidosc., Ind.-Driv. Digit. Transf. (ITU K)*, 2020, pp. 1–8.

[12] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.

[13] R. Antonello et al., "Deep packet inspection tools and techniques in commodity platforms: Challenges and trends," *J. Netw. Comput. Appl.*, vol. 35, pp. 1863–1878, Nov. 2012.

[14] A. Panchenko et al., "Website fingerprinting at Internet scale," in *Proc. NDSS*, 2016, pp. 1–15.

[15] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, pp. 63–78, 2018.

[16] K. Al-Naami et al., "Adaptive encrypted traffic fingerprinting with bi-directional dependence," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 177–188, doi: 10.1145/2991079.2991123.

[17] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Security Informat. (ISI)*, 2017, pp. 43–48.

[18] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, Toronto, Canada, 2018, pp. 1928–1943.

[19] S. Bhat, D. Lu, A. Kwon, and S. Devadas, "Var-CNN: A data-efficient Website fingerprinting attack based on deep learning," in *Proc. Priv. Enhanc. Technol.*, vol. 2019, no. 4, pp. 292–310, Jul. 2019. doi: 10.2478/popets-2019-0070.

[20] M. Lotfollahi, M. Jafari Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," 2017, *arXiv:1709.02656*.

[21] Y. Luo, M. He, X. Wang, and L. Jin, "Network flow detection of semantic relationship between flow and byte," in *Proc. IEEE 8th Int. Conf. Big Data Comput. Service Appl. (BigDataService)*, 2022, pp. 179–180.

[22] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021, doi: 10.1109/TIFS.2021.3050608.

[23] T.-L. Huoh, Y. Luo, P. Li, and T. Zhang, "Flow-based encrypted network traffic classification with graph neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1224–1237, Jun. 2023, doi: 10.1109/TNSM.2022.3227500.

[24] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapè, "MIMETIC: Mobile encrypted traffic classification using multimodal deep learning," *Comput. Netw.*, vol. 165, Dec. 2019, Art. no. 106944, doi: 10.1016/j.comnet.2019.106944.

[25] I. Guarino, G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapè, "Contextual counters and multimodal deep learning for activity-level traffic classification of mobile communication apps during COVID-19 pandemic," *Comput Netw*, vol. 219, Dec. 2022, Art. no. 109452, doi: 10.1016/j.comnet.2022.109452.

[26] X. Wang, S. Chen, and J. Su, "App-Net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE Conf. Comput. Commu. Workshops (INFOCOM WKSHPS)*, 2020, pp. 424–429, doi: 10.1109/INFOCOMWKSHPS50562.2020.9162891.

[27] X. Wang, H. Wang, and D. Yang, "Measure and improve robustness in NLP models: A survey," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist., Human Lang. Technol.*, 2022, pp. 4569–4586.

[28] A. Habibi Lashkari, G. Draper Gil, M. Mamun, and A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.

[29] J. Gong and T. Wang, "Zero-delay lightweight defenses against Website fingerprinting," in *Proc. 29th USENIX Conf. Security Symp.*, 2020, pp. 1–18.

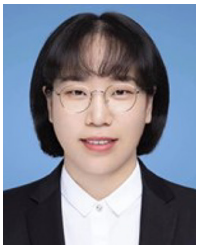
[30] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient Website fingerprinting defense," 2016, *arXiv:1512.00524*.

[31] N. Mathews, J. K. Holland, S. E. Oh, M. S. Rahman, N. Hopper, and M. Wright, "SoK: A critical evaluation of efficient Website fingerprinting defenses," in *Proc. IEEE Symp. Security Privacy (SP)*, 2023, pp. 969–986, doi: 10.1109/SP46215.2023.10179289.

[32] X. Deng et al., "Robust multi-tab Website fingerprinting attacks in the wild," in *Proc. IEEE Symp. Security Privacy (SP)*, 2023, pp. 1005–1022, doi: 10.1109/SP46215.2023.10179464.

[33] Z. Yao, J. Ge, Y. Wu, X. Lin, R. He, and Y. Ma, "Encrypted traffic classification based on Gaussian mixture models and hidden Markov models," *J. Netw. Comput. Appl.*, vol. 166, Sep. 2020, Art. no. 102711.

[34] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. Int. Conf. Inf. Syst. Security Privacy*, 2017, pp. 253–262.
[35] N. Moustafa, M. Ahmed, and S. Ahmed, "Data analytics-enabled intrusion detection: Evaluations of ToN_IoT Linux datasets," in *Proc. IEEE 19th Int. Conf. Trust, Security Privacy Comput. Commun. (TrustCom)*, 2020, pp. 727–735.
[36] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, 2017, pp. 712–717.
[37] "The TON_IoT datasets," Datasets, UNSW Sydney. Accessed: Mar. 15, 2023. [Online]. Available: https://research.unsw.edu.au/projects/toniot-datasets
[38] Y. Zeng, Z. Qi, W. Chen, and Y. Huang, "TEST: An end-to-end network traffic classification system with spatio-temporal features extraction," in *Proc. IEEE Int. Conf. Smart Cloud (SmartCloud)*, 2019, pp. 131–136.
[39] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1171–1179.
[40] T. van Ede et al., "FlowPrint: Semi-supervised mobile-App fingerprinting on encrypted network traffic," in *Proc. ISOC Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020, pp. 1–18.
[41] Z. Wu, W. Li, L. Liu, and M. Yue, "Low-rate DoS attacks, detection, defense, and challenges: A survey," *IEEE Access*, vol. 8, pp. 43920–43943, 2020.
[42] S. Li, H. Guo, and N. Hopper, "Measuring information leakage in Website fingerprinting attacks and defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1977–1992.
[43] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep./Oct. 2022.
[44] "Deeppacket." GitHub.com. Accessed: Apr. 15, 2023. [Online]. Available: https://github.com/KimythAnly/deeppacket
[45] P. Sirinam et al. "df." GitHub.com. Accessed: Apr. 26, 2023. [Online]. Available: https://github.com/deep-fingerprinting/df
[46] T. Wang et al. "Wangknn-dataset." Accessed: May 7, 2023. [Online]. Available: https://github.com/kdsec/wangknn-dataset
[47] S. Bhat et al. "Var-CNN." Accessed: May 8, 2023. [Online]. Available: https://github.com/sanjit-bhat/Var-CNN
[48] T. van Ede et al. "FlowPrint." Accessed: Apr. 15, 2023. [Online]. Available: https://github.com/Thijsvanede/FlowPrint



Xiaojuan Wang received the Ph.D. degree in electronic science and technology from the University of Beijing University of Posts and Telecommunications, where she is currently an Associate Professor with the School of Electronic Engineering. Her research interests include deep learning, complex networks, and human gesture recognition.

Zikui Lu is currently pursuing the Ph.D. degree in electronic science and technology with the Beijing University of Posts and Telecommunications. His research interests include deep learning and traffic analysis.

Xinlei Wang is currently pursuing the Ph.D. degree in electronic science and technology with the Beijing University of Posts and Telecommunications. Her research interests include deep learning and computer network security.

Mingshu He (Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022, where he is currently doing research with the School of Cyberspace Security. His research interests include network security, anomaly detection, and machine learning.

Xiaojun Wang received the B.Eng. degree in computer and communications and the M.Eng. degree in computer applications from the Beijing University of Posts and Telecommunications (BUPT) in 1984 and 1987, respectively, and the Ph.D. degree from Staffordshire University, England. He was an Assistant Lecturer/Lecturer with BUPT from 1987 to 1989. He joined the School of Electronic Engineering, Dublin City University in 1992, where he is currently an Associate Professor. His research interests include network security, energy-efficient networking, and hardware acceleration of artificial intelligence and cryptography algorithms. He received a Sino-British Technical Co-operation Training Award in 1989.